Building RAG Systems

A Comprehensive Guide to Retrieval Augmented Generation

Table of Contents

Part I: Understanding RAG Systems

1. The Core Components of RAG
2. Data Collection and Ingestion
3. Embeddings and Vector Databases
4. The Retrieval Process
5. Generation and Prompting
6. Building the Full Flow
7. Enhancements and Deployment

Part II: Implementing RAG with Prompt Engineering

8. The Master Prompt Blueprint
9. Language-Neutral Implementation Templates
10. Specific Stack Templates (Spring Boot & Angular)
11. Iterative Refinement & Testing Strategies
12. Security, Compliance, and Troubleshooting

PART I
Understanding RAG Systems

This section outlines the foundational knowledge required to understand Retrieval Augmented Generation (RAG). It covers the journey from raw data to a functioning answer engine.

1: The Core Components of RAG

✓ **Definition:** RAG (Retrieval Augmented Generation) = Retriever + LLM. It is the process of adding external knowledge (your documents, database, PDFs, etc.) that the Language Model did not train on.

Why Build a RAG System?

RAG solves two fundamental problems inherent in static Large Language Models (LLMs):

1. Hallucinations: LLMs can confidently invent facts. RAG forces answers to be based on retrieved evidence.
2. Updatable Knowledge: Retraining models is expensive. RAG allows you to update the knowledge base instantly by adding documents to the retrieval system without touching the model weights.

2: Data Collection and Ingestion

Step 1: Collect Your Data

The first step is gathering the sources you want the system to utilize. Common data sources include:

- PDF Files and Word Documents
- Website pages (HTML)
- Database tables (SQL)
- Emails, CSVs, and Excel sheets

Step 2: Document Loading

You must load these documents into memory to extract clean text. While every programming language has libraries for this (e.g., LangChain loaders for Python, LangChain4j for Java), the logical flow is universal:

```
// Pseudo-code:
load(document) -> read content -> extract clean text -> return text
```

Step 3: Chunking the Text

LLMs have context limits and cannot process massive documents at once. We break text into "chunks".

Typical Chunk Size: 300–1000 tokens
Overlap: 50–150 tokens (to preserve context across boundaries)

The general logic is:

```
chunks = split_text(text, chunk_size=500, overlap=100)
```

3: Embeddings and Vector Databases

Step 4: Generate Embeddings

Each text chunk must be converted into a vector—a list of numbers representing semantic meaning. This enables similarity search.

Common embedding models include OpenAI, Azure OpenAI, AWS Titan, Google Embeddings, or local models like BGE/Instructor.

Ideally, the process is:

```
vector = embedding_model.embed(chunk_text)
```

Step 5: Store in a Vector Database

Vectors require specialized storage for efficient retrieval. You may choose between local solutions (FAISS, ChromaDB) or cloud solutions (Pinecone, Azure AI Search, Weaviate, Milvus).

Storage Operation:

```
vector_store.save(vector, metadata={chunk_text, source, page_number})
```

4: The Retrieval Process

When a user asks a question, the system must find the relevant information before generating an answer. This is the "Retrieval" in RAG.

⭐ Retrieval Logic:

3. Convert user question → embedding vector.
4. Search vector DB for vectors mathematically similar to the question vector.
5. Return the most relevant chunks (Top-K).

```
// Neutral Pseudo-code
query_vector = embed(user_question)
relevant_chunks = vector_store.similarity_search(query_vector, top_k=3)
```

5: Generation and Prompting

Building the RAG Prompt

Once you have the user's question and the relevant chunks, you construct a prompt. This prompt instructs the LLM to use the provided context.

PROMPT STRUCTURE:

```
"Use the following context to answer:

{retrieved_chunks}

User question:
{query}"
```

Calling the LLM

The final step is sending this prompt to the model (Generation). The result is a grounded answer:

```
answer = llm.generate(prompt)
```

6: Building the Full Flow

A complete RAG pipeline integrates all previous steps into a seamless flow:

6. User asks a question.
7. System embeds the question.
8. Vector DB performs similarity search.
9. System retrieves relevant text chunks.
10. System constructs the RAG prompt (Chunks + Question).
11. LLM generates the answer.
12. System returns the answer to the user.

7: Enhancements and Deployment

Optional Enhancements

- Conversational Memory: Allows for multi-turn chat history.
- Citations: References the specific document or page number used.
- SQL + RAG Hybrid: Uses SQL for structured queries and RAG for unstructured text.
- Guardrails: Prevents harmful or irrelevant prompts.
- Caching: Speeds up responses for repeated questions.

Monitoring & Evaluation

Crucial metrics to monitor include answer correctness, hallucination rates, and retrieval precision. Tools often used include human evaluation and automated test queries.

PART II
Implementing RAG with Prompt Engineering

This section provides actionable "Blueprints" that you can feed into AI code assistants (like GitHub Copilot, Amazon Q, or ChatGPT) to generate production-ready code for your RAG system.

8: The Master Prompt Blueprint

Use this prompt to initialize your project with a senior AI pair programmer. It sets the architecture, requirements, and tech stack.

```
You are my senior AI pair-programmer. Build a production-ready RAG (Retrieval
Augmented Generation) web app with the following requirements.
Ask me clarifying questions if any requirement is ambiguous before generating
code.

# Goal
A chat-style UI where users ask questions, and the backend performs RAG over my
documents (PDF/Docx/HTML) and optionally SQL tables, then responds with
grounded answers including citations.

# Architecture (flexible to my stack)
- Frontend: Single-page app with chat UI, streaming responses, source
citations, and message-level feedback (👍/👎).
- Backend: REST/GraphQL API with endpoints: /ingest, /chat, /health, /metrics.
- RAG pipeline: Load docs -> Chunk -> Embed -> Vector Store -> Retrieve ->
Prompt -> LLM.

# Functional Requirements
- Document ingestion UI: drag-and-drop upload, progress, and index status.
- Citations: show per-chunk source (filename, page/section, score).
- Config: environment-based config for keys, model names, top_k, chunk params.
- Security: JWT-based auth, CORS, rate limiting, prompt injection guardrails.

# Ask Before Building
- Preferred stack (language, web framework, vector store)?
- Deployment target?
- Do you want SQL-hybrid retrieval?

Proceed by proposing an implementation plan and folder structure first. Then
wait for my approval.
```

9: Language-Neutral Implementation Templates

Once the master plan is approved, use these templates to build specific components.

A. Project Setup Scaffold

Create a new RAG application scaffold.

```
Stack: Backend [Language/Framework], Frontend [Framework], Vector Store
[Choice].

Generate:
1) Folder structure and key files.
2) .env.example, config loader, secrets handling.
3) Dockerfile(s) + docker-compose.yml.

Wait for my approval before generating full code.
```

B. RAG Pipeline Module

Implement a RAG module with these interfaces:

```
- DocumentLoader: load & clean text.
- TextSplitter: chunk(text, chunk_size, overlap).
- Embedder: embed(text[]) -> vectors.
- VectorStore: upsert(chunks, vectors), query(vector).
- PromptBuilder: build(system, instructions, query, contexts).

Include unit tests for each component.
```

C. Endpoints & Contracts

Design REST endpoints:

```
- POST /ingest (multipart upload)
- POST /chat {question, top_k} -> {answer, citations}
- GET /health, GET /metrics

Return a consistent error schema. Add OpenAPI/Swagger.
```

10: Specific Stack Templates (Spring Boot & Angular)

If you are using Java Spring Boot and Angular, use this specialized prompt.

```
Build a Spring Boot (Java 21) + Angular 17 RAG application.

Backend:
- Spring Boot modules: web, security (JWT), validation.
- RAG core: Clean architecture (domain/services/adapters). Use LangChain4j
where reasonable.
- Vector store: start with FAISS (local) adapter.
- SQL-hybrid: add optional JDBC access to MySQL.
- Endpoints: /ingest, /chat, /health.

Frontend (Angular):
- Chat UI with streaming, citations, copy, and feedback.
- Ingestion screen with upload progress.

Deliverables:
- Project skeleton, runnable locally (docker compose).
- Scripts: dev.sh, test.sh.
- README with architecture diagram.

Ask me before choosing embedding/LLM providers.
```

11: Iterative Refinement & Testing Strategies

Iterative Refinement Prompts

Use these prompts after the initial code generation to refine the system.

- Plan Review: "Summarize your proposed architecture and data flow. Call out trade-offs."
- Component-by-Component: "Implement the TextSplitter with tests first. Use TDD."
- Prompt Budgeting: "Implement token budgeting to select minimal highest-scoring chunks within limits."

Testing & Observability

Unit & Integration Tests:

Create unit tests for TextSplitter and Embedder. Create integration test for /chat exercising ingestion -> retrieval -> generation with a stub LLM.

Metrics & Tracing:

Add counters: retrieval_latency_ms, tokens_in/out, top_k_used. Expose /metrics for Prometheus.

12: Security, Compliance, and Troubleshooting

Security & Guardrails

Security Prompt Template:

Add:

- JWT auth with roles (ADMIN ingest, USER chat).
- Input validation: max upload size, question length.
- Prompt-injection defenses: strip HTML/URLs, neutralize instructions in context.
- Audit logging of access to /chat.

Troubleshooting Prompts

If your RAG system behaves poorly, use these prompts to diagnose:

- Low Relevance: "Diagnose poor retrieval. Print top 10 terms in query and top 5 retrieved chunks. Suggest changes to chunk size/overlap."
- Hallucinations: "Add an abstain mechanism: if no chunk has score >= threshold, return 'insufficient context'."
- Latency: "Profile the pipeline. Output breakdown of time in load->split->embed->retrieve->LLM."

Summary: Your RAG Roadmap

Stage	Goal	Key Action
1. Basics	Understand Concepts	Learn RAG = Retriever + LLM
2. Gather Data	Source Material	Collect PDFs, SQL, Text
3. Ingestion	Process Data	Load, Clean, Chunk, Embed
4. Vector DB	Storage	Index vectors in FAISS/Pinecone
5. Retrieval	Search	Query vectors → Top-K Chunks
6. Generation	Answer	Build Prompt → Call LLM
7. Deployment	Production	Secure, Monitor, and Deploy